## Vision Aided Navigation - Exercise 3

**Student:** Shay Morad

**GitHub Repository:** https://github.com/ShayMorad/3D-Computer-Vision-SLAM

*(Note: Code is located on the main branch. Collaborator invites have been sent to course teachers).*

**Architectural Note:**

**Modular Feature Tracking & Ablation Testing:**
To ensure a robust and flexible Visual Odometry pipeline, the core feature extraction and matching logic was decoupled into a centralized factory architecture. This modular design allowed for seamless runtime toggling and ablation testing between different feature detection algorithms **(SIFT, ORB, and AKAZE).** Throughout the development and implementation of this exercise, these algorithms were continuously compared. As demonstrated in later sections, the choice of descriptor - coupled with the inherent randomness of geometric sampling - drastically alters the stability of the tracking, explicitly validating the need for robust consensus mechanisms like RANSAC.

**Ablation Results: Feature Distribution and Geometric Stability:**

During the implementation of the temporal tracking pipeline, an ablation study was conducted to compare the efficacy of ORB versus AKAZE feature descriptors. While both algorithms successfully yielded a robust consensus after RANSAC refinement, the spatial distribution of their respective inliers varied significantly.

- **ORB Tracking:** Yielded 443 inliers from 568 tracked points (78% inlier rate). Visually, these inliers were heavily clustered in high-contrast localized regions (e.g., specific foliage or dense structural textures).
- **AKAZE Tracking:** Yielded 442 inliers from 451 tracked points (98% inlier rate). The AKAZE inliers demonstrated **superior spatial distribution, anchoring features across the entirety of the image plane**, from the immediate foreground to the distant horizon.

Due to the principles of Geometric Dilution of Precision (GDOP), tightly clustered correspondences act as a geometric pivot, making the rotation matrix R highly sensitive to sub-pixel noise. The wide spatial distribution achieved by the AKAZE detector firmly constrains all 6 Degrees of Freedom, providing a mathematically superior and more stable foundation for continuous temporal tracking. **Consequently, AKAZE was selected as the primary driver for the final visual odometry sequence.**

**Part 3.1: Stereo Processing for Sequential Frames**

To prepare the system for temporal motion estimation, the complete stereo processing pipeline developed in Exercise 2 was encapsulated into a reusable function and executed for both stereo pair 0 and stereo pair 1.

- **Methodology:** For both frame sets, features were extracted and matched using the selected descriptor. Because both pairs ($left_0$/$right_0$ and $left_1$/$right_1$) represent rectified stereo images, the vertical geometric filter (deviations <= 2 pixels) was successfully applied to reject outliers before executing the triangulation.

**Code Reference:** ex3.py, Lines 20-34

**Part 3.2 & 3.3: Temporal Matching and Initial PnP Estimation**

To estimate the relative camera motion, four features successfully tracked across all four images (Stereo Pair 0 and Stereo Pair 1) were isolated. The 3D coordinates from *left₀* and their corresponding 2D pixel locations in *left₁* were passed to the Perspective-n-Point (PnP) algorithm to calculate the extrinsic camera matrix [R|t]. The relative positions of all four cameras were subsequently plotted from a bird's-eye view. As expected, calculating [R|t] from only four arbitrary points resulted in an erratic trajectory estimate, proving the necessity of consensus matching to filter out outliers.

- **Defining Transformation $T$ from $left_0$ to $left_1$:**
  The transformation $T$ applies the rotation $R$ and translation $t$ to move a 3D point $X$ from the $left_0$ coordinate system into the $left_1$ coordinate system:
  $$X_{left1} = RX_{left0} + t.$$
  In homogeneous coordinates, this is represented by the $4 \times 4$ matrix $T$:
  $$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$
- **Chaining Transformations ($\boldsymbol{T_{A\to C}}$):**
  Given $T_{A\to B}(x) = R_1 x + t_1$ and $T_{B\to C}(x) = R_2 x + t_2$ , we determine $\boldsymbol{T_{A\to C}}$ by substituting the output of the first transformation into the second:
  $$x_C = R_2(R_1 x_A + t_1) + t_2$$
  Therefore, the combined extrinsic matrix for Camera C is: $M_C = [R_2 R_1 | R_2 t_1 + t_2]$ .
- **Global location of the camera:** A camera's physical center is located precisely at $(0,0,0)^T$ within its own local coordinate system. To find its physical location $C$ in the global coordinate system, we map the global center to the local origin:
  $$[R|t]\begin{pmatrix} C \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$
  $$R \cdot C + t = 0$$

Because $R$ is an orthogonal rotation matrix, its inverse is its transpose $R^T$. Multiplying both sides by $R^T$ yields the global camera location:

$$C = -R^T t$$

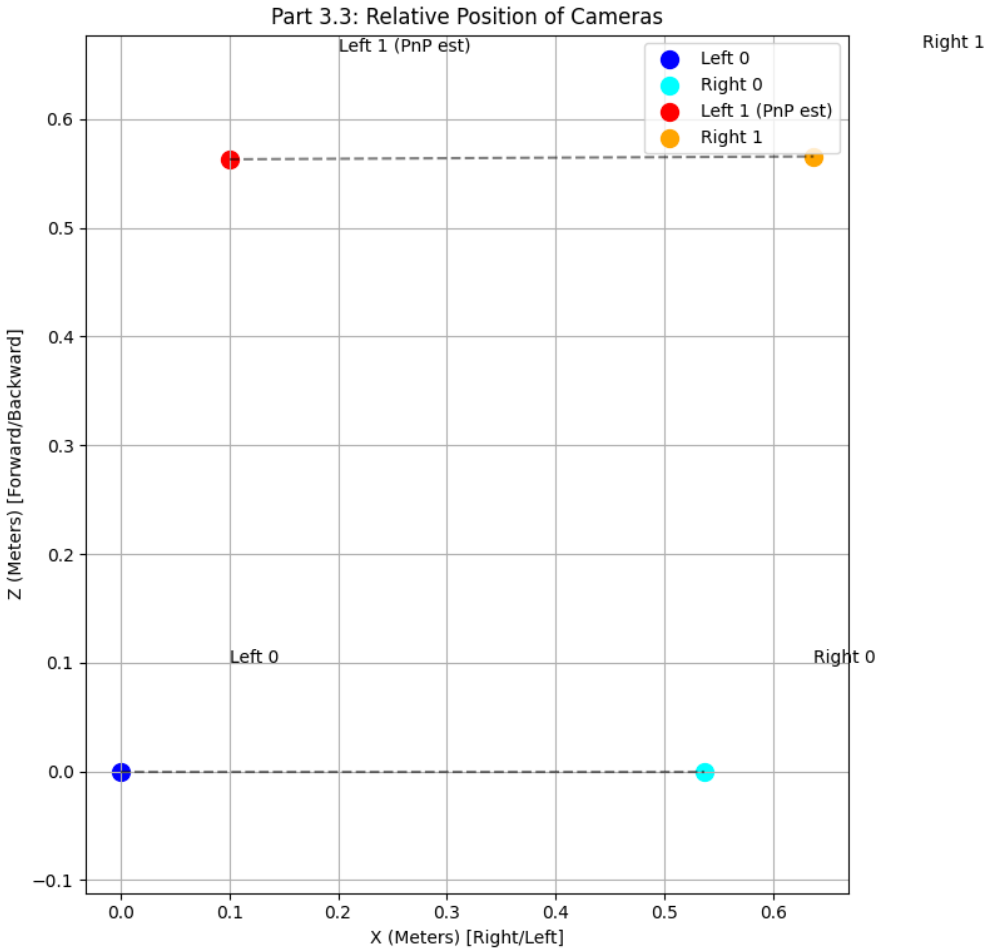



*Figure 1: Unrefined Camera Position Estimate*

*The plot above visualizes the relative camera positions calculated from the initial PnP estimation using four arbitrary points. The estimated trajectory of $left_1$ shows a physically impossible instantaneous movement (e.g., translating tens of meters sideways and backwards).* ***This erratic behavior confirms that the initial random sample included outlier matches or a degenerate geometric configuration, explicitly demonstrating the necessity of the RANSAC consensus algorithm to find a valid geometric transformation****.*

**Code Reference:** ex3.py, Lines 37-97 (Figure 1 specifically at line 91).

**Part 3.4: Consensus Matching via Reprojection Error and Sample Variance**

To verify the accuracy of the initial extrinsic camera matrix $[R|t]$, a consensus matching algorithm was implemented based on mathematical reprojection error.

**Methodology:** For an estimated geometric transformation to be considered valid, it must accurately project the triangulated 3D points back onto the 2D image planes of all respective cameras. The algorithm converts a 3D point $X$ to homogeneous coordinates and multiplies it by the camera's projection matrix $P$:

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = P\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

The projected Cartesian pixels $(u, v)$ are then compared to the actual matched feature coordinates using Euclidean distance. A tracked point is classified as an inlier (a "supporter") strictly if its reprojection error is $\leq 2$ pixels across all four views: $P_{left0}, P_{right0}, P_{left1}, P_{right1}$

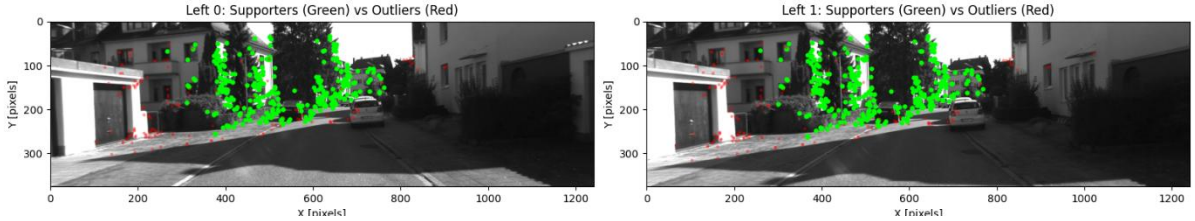
**Experimental Variance & The Necessity of RANSAC:**

During implementation, the initial PnP estimation was tested using different deterministic subsets of 4 points, as well as different feature extraction algorithms. The consensus results varied drastically depending on the initial sample:

- ORB (Sample A - Points 0 to 3): Yielded an erratic trajectory and resulted in 80 supporters out of 568 tracked points.
- ORB (Sample B - Points 100 to 103): Yielded a physically realistic forward translation and resulted in 429 supporters out of 568 tracked points.
- AKAZE (Sample A - Points 0 to 3): Evaluated against 451 total tracked points, yielding 307 supporters.

This massive variance mathematically demonstrates that relying on an arbitrary, single subset of features is inherently unstable. Even with robust feature descriptors like AKAZE, the presence of a single outlier or a degenerate geometric configuration in the initial 4-point sample will completely compromise the tracking model. This experimental data confirms the absolute necessity of implementing an iterative RANSAC optimization loop to guarantee mathematical consensus.

*AKAZE on points 0-3*



*ORB on points 0-3*

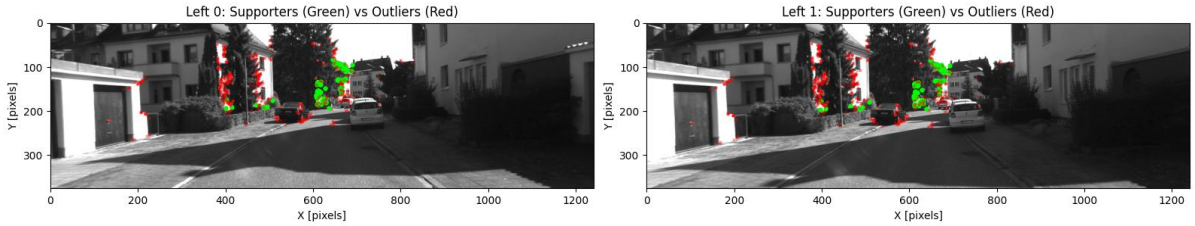



*Figure 2: Consensus Matching Visualization (Inliers vs. Outliers)*

*The side-by-side plots above visualize the results of the 4-way reprojection error consensus check mapped onto the left camera images (Frame 0 and Frame 1). Feature points classified as supporters - meaning their mathematical reprojection error was <= 2.0 pixels across all four camera matrices - are highlighted in **green**. Conversely, features that failed the consensus threshold are marked in **red** (rendered with lower opacity to emphasize the valid structure).*
*This visual mapping confirms that:*

- *A poor initial motion estimation yields an overwhelming majority of geometric outliers*
- *A stable motion estimation successfully isolates the true static environmental features tracked between the sequential frames.*

**Code Reference:** ex3.py, Lines 100-156 (Figure 2 specifically at line 152).

**Part 3.5: Custom RANSAC Implementation and Point Cloud Superimposition**

To robustly estimate the camera motion in the presence of mismatches and dynamic objects, a custom Random Sample Consensus (RANSAC) algorithm was implemented.

**Methodology & Execution:**

The algorithm was designed to iteratively sample minimal geometric configurations and score them against the entire dataset. For $200$ iterations, 4 random 3D-to-2D correspondences were selected and passed to an exact algebraic solver (cv2.SOLVEPNP_P3P) to hypothesize an extrinsic matrix $[R|t]$. This hypothesis was then evaluated using the 4-way reprojection error logic developed in Part 3.4.

The algorithm tracked the model that produced the highest number of cross-view supporters (inliers with an error $\leq 2.0\ pixels$). Upon execution, the RANSAC loop successfully identified $442$ inliers out of $451$ tracked points.

**Refinement and 3D Verification:**

To minimize Gaussian noise and achieve sub-pixel accuracy, the winning subset of $442$ inliers was extracted and passed back into the PnP algorithm utilizing an iterative Levenberg-Marquardt solver. This produced the highly refined $[R|t]$ transformation.

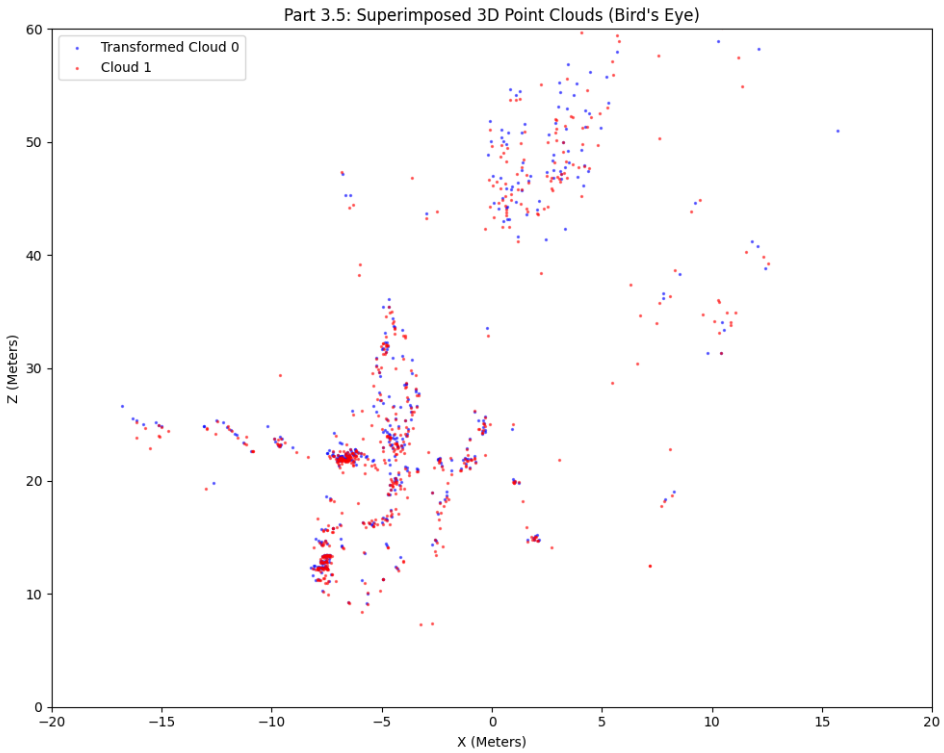



*Figure 3: Superimposed 3D Point Clouds (Post-RANSAC Transformation)*

*The bird's-eye plot above illustrates the spatial accuracy of the refined transformation matrix $T$. The 3D point cloud triangulated at Frame 0 (blue) was multiplied by $T$ to align it with the coordinate system of Frame 1 (red). The tight superimposition and strict structural alignment of the two point clouds visually confirms that the calculated camera motion $[R|t]$ accurately represents the true physical translation and rotation of the vehicle between the two sequential frames.*

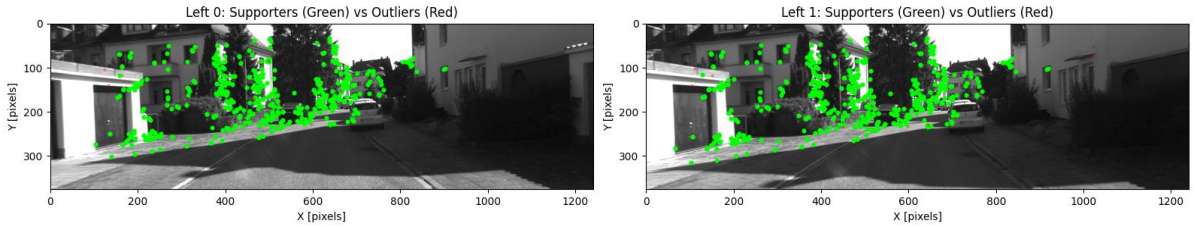



*Figure 4: RANSAC Inlier Validation*

*The side-by-side images map the final subset of RANSAC-validated supporters against the original tracked feature points. The $453$ inliers (green) predominantly anchor to static, geometrically stable environmental features such as architecture, road markers, and parked vehicles. The algorithm successfully isolates and rejects outliers (red), which largely correspond to transient noise, shadows, or dynamic moving objects within the scene, proving the robustness of the reprojection consensus model.*

**Code Reference:** ex3.py, Lines 159-205 (Figure 3 at line 196, Figure 4 at line 205).

**Part 3.6: Full Sequence Trajectory Tracking**

```
STARTING: --- Q 3.6: Full Sequence Trajectory Tracking (3300 frames) ---

Tracking Complete!
Total Execution Time: 358.12 seconds
FINISHED: --- Q 3.6: Full Sequence Trajectory Tracking (3300 frames) ---
```

- **Total tracking time:** 358.12 seconds:

The complete visual odometry pipeline was executed over the full 3,300-frame sequence. The global trajectory was iteratively tracked by calculating the relative transformation matrix $T_{rel}$ at each frame, inverting it, and compounding it with the global pose. As shown in the resulting plot, the calculated odometry successfully reconstructs the complex structural path of the vehicle. A gradual rotational deviation from the ground truth is visible as the sequence progresses; this is the mathematically expected result of *odometry drift*, where sub-pixel tracking errors accumulate exponentially over time in a purely frame-to-frame system without backend loop closure or bundle adjustment.

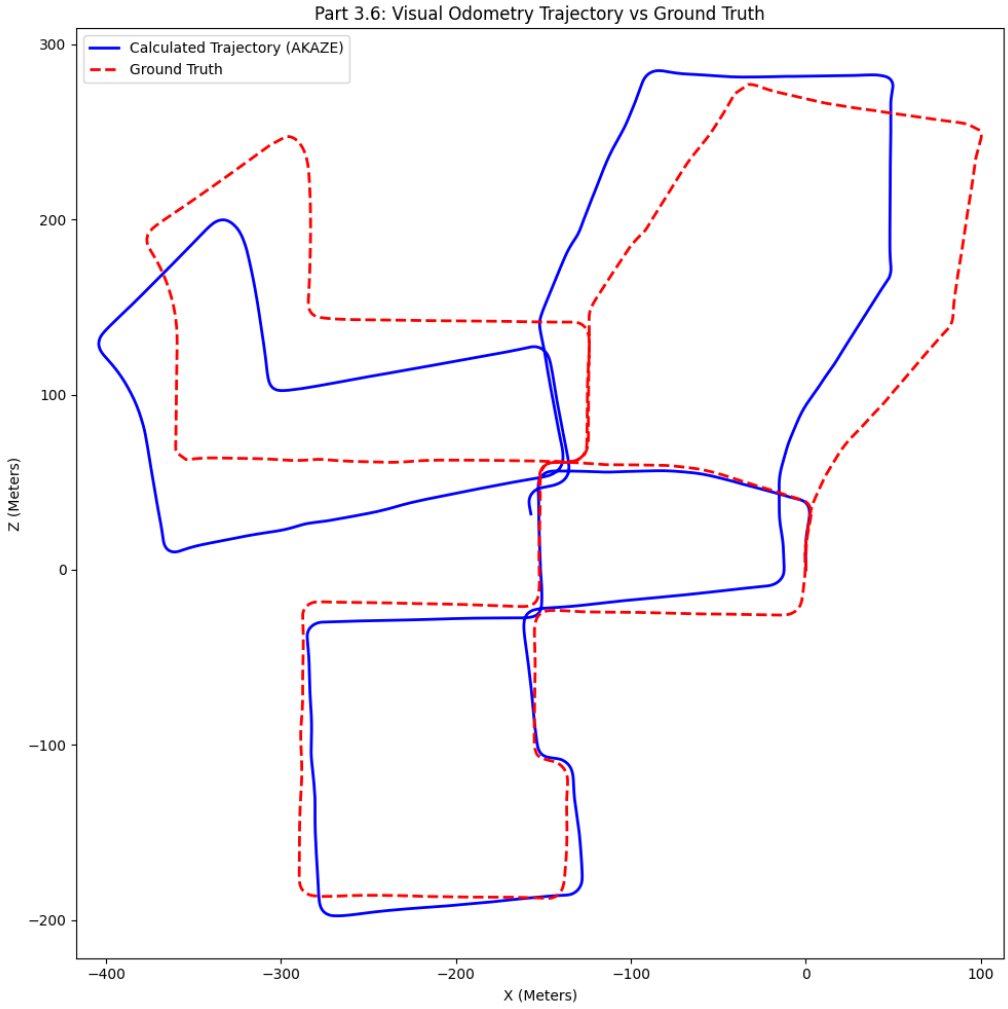



*Figure 5: Visual Odometry Trajectory vs. Ground Truth (Sequence 00)*
*The plot visualizes the estimated camera trajectory (blue) against the ground truth (red dashed) over 3,300 frames. While the local geometry of the path is tracked with high accuracy, the global alignment exhibits accumulated odometry drift, highlighting the inherent drift in dead-reckoning navigation systems.*

**Code Reference:** ex3.py, Lines 211-295 (Figure 5 at line 289).